

SQL injection is a method of attack used against unsecure SQL databases where the attacker uses certain strings to attempt to bypass normal checks, including logging in with elevated privileges, causing data leaks, or even modifying or deleting the information that is on the database. In the provided demo, we had two different methods that were used to “login” to the mock database, one secure, and one insecure. The secure one works by comparing directly with the information in the database, looking for a match to both the username and password. This method, however, would not be particularly feasible on a real database, as it would require the transmission of a large amount of data for the client to perform the comparison, and would take a long time. The insecure method did a direct concatenation, without any checks or sanitization to prevent SQLi, so a simple tautology works to login to the database.

I also created a simple real database on a server I have through Azure (it is a free account I have that I use for school projects, such as this), where I created a simple table of 3 users, each with a password, and a “secret” to indicate that they were logged in. For the API’s to connect to the database, there are two methods presented in the program – Secure Login and Insecure Login. The secure method uses a Java class called “Prepared Statement”, which sanitizes inputs to prevent SQLi attacks, by parametrizing the inputs, rather than concatenation. The insecure method again uses simple concatenation, which still allows for an SQLi attack, providing false logins, as well as allowing for data leaks, including the whole table. In the interface, I do provide a couple of attack strings that work for various purposes.

If you would like to explore the app I made, it should run on any system that has Apache Maven installed with CLI input ‘mvn javafx:run’.

```
[VULNERABLE] Executing: "Select * From users Where 'username'='alice' AND 'password'='password1';"  
[VULNERABLE] Executing: "Select * From users Where 'username'='alice' AND 'password'='admin' OR '1'='1';"  
HACK SUCCESSFUL  
[SAFE] Checking with parameters - query: "SELECT * FROM users WHERE 'username'='alice' AND 'password'='admin' OR '1'='1';"  
Failed to log in  
[SAFE] Checking with parameters - query: "SELECT * FROM users WHERE 'username'='alice' AND 'password'='password1';"  
Safe Login Successful  
[SAFE] Checking with parameters - query: "SELECT * FROM users WHERE 'username'='alice' AND 'password'='hunter2';"  
Failed to log in
```

My project meets all requirements.

Screenshots from my mini-app:

SQLi Tester and Demonstration

Username	Password
admin	admin123
alice	password1
bob	hunter2

SQLi Tester and Demonstration

This simple app demonstrates SQL injection attacks, as well as a way to prevent them in Java. The above table shows the default usernames and passwords that are in the database table. Clicking "Secure Login" will run the SQL query through PreparedStatement, which parametrizes the input, preventing most SQLi attacks. However, "Insecure Login" concatenates the input to the query, resulting in insecure strings, allowing for a wide variety of SQLi attacks. All users have a server-side secret that shows on successful login (or SQLi attack).

Username:

Password:

Secure Login

Insecure Login

Tautology Attacks: ' OR '1'='1'; -- (in password, will also dump all rows)
admin' -- (in username)

Data Leak Attack: ' OR 1=1 UNION ALL SELECT username, passwd, secret FROM dbo.Users --

In addition to the above attacks, several other attacks work as well, including inserting/deleting rows, changing values, and even deleting the whole table (closing and reopening the app will re-generate the default table).

Username:

Password:

You have successfully conducted a SQLi attack and logged in with the following command:

```
SELECT username, passwd, secret FROM dbo.Users WHERE username='admin' AND passwd='' OR 1=1; --
```

And the database returned the following:

```
username | passwd | secret
-----
admin | admin123 | ADMIN-SECRET
alice | password1 | TAGGED
bob | hunter2 | BOB-SECRET
```

[Return to main](#)

You have successfully conducted a SQLi attack and logged in with the following command:

```
SELECT username, passwd, secret FROM dbo.Users WHERE username=""; UPDATE dbo.Users SET secret='ALICE-SECRET' WHERE username='alice'; -- ' AND passwd=";
```

You have successfully conducted a SQLi attack and logged in with the following command:

```
SELECT username, passwd, secret FROM dbo.Users WHERE username=" OR 1=1; --' AND passwd=";
```

And the database returned the following:

username	passwd	secret
admin	admin123	ADMIN-SECRET
alice	password1	ALICE-SECRET
bob	hunter2	BOB-SECRET

The above pictures show tautology attack, data dump, and changing the information through injection.

You have not logged in with the following SQL command using the secure login:

```
[Prepared] SELECT username, ISNULL(CAST(secret AS VARCHAR(100)), '(null)') AS secret FROM dbo.Users WHERE username=? AND passwd=? | params: [username='admin', passwd=" OR 1=1; --"] // Injection blocked by parameters
```

And the database returned the following:

(no rows)

This shows how attempting the injection through the secure login fails, and that the system catches the attempted injection and fails it.

You have not logged in with the following SQL command using the secure login:

```
[Prepared] SELECT username, ISNULL(CAST(secret AS VARCHAR(100)), '(null)') AS secret FROM dbo.Users WHERE username=? AND passwd=? | params: [username='admin', passwd='pass']
```

And the database returned the following:

(no rows)

By comparison, if you use the wrong password, this is the result.

You have successfully logged in with the following SQL command using the secure login:

```
[Prepared] SELECT username, ISNULL(CAST(secret AS VARCHAR(100)), '(null)') AS secret FROM dbo.Users WHERE username=? AND passwd=? | params: [username='admin', passwd='admin123']
```

And the database returned the following:

username	secret
admin	ADMIN-SECRET

A successful, secure login.

One limitation I ran into on the injections is space. While I could have made it larger, the output is always limited currently to 3 rows. So, if you add rows, you have to use those rows specifically. Also, due to how my database is hosted, it sleeps after 5 minutes of inactivity. If this happens, any ping to the server will wake it back up, however, that ping will return an error.